STAT 508 - HW5

Ana Camargo

2023-04-21

Chapter 12

```
library(ISLR2)
```

Problem 2

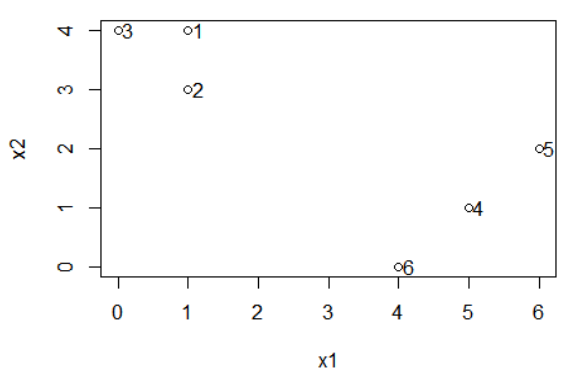
See end of document (scanned)

Problem 3

```
# creating table of observations as data frame
c1 <- c(1:6)
c2 <- c(1,1,0,5,6,4)
c3 <- c(4,3,4,1,2,0)
T <- cbind.data.frame(Obs. = c1, X1 = c2, X2 = c3)
T

##   Obs. X1 X2
## 1    1  1  4
## 2    2  1  3
## 3    3  0  4
## 4    4  5  1
## 5    5  6  2
## 6    6  4  0
```

a) Plot the observations

```
plot(T$X1,T$X2, xlab="x1", ylab="x2")
text(T$X1+0.15,T$X2,T$Obs.)
```



b) Randomly assign a cluster label to each observation. You can use the sample() command in R to do this. Report the cluster labels for each observation.

```
cl <- rep(NA,6)
set.seed(3)
for (i in 1:6){
  cl[i] <- sample(1:2,1)
}
T2 <- cbind.data.frame(T, cluster = cl)
T2

##   Obs. X1 X2 cluster
## 1    1  1  4       1
## 2    2  1  3       2
## 3    3  0  4       2
## 4    4  5  1       1
## 5    5  6  2       2
## 6    6  4  0       2
```

c) Compute the centroid for each cluster.

```
cl1_x1 <- mean(T2$X1[T2$cluster==1])
cl1_x2 <- mean(T2$X2[T2$cluster==1])
cl2_x1 <- mean(T2$X1[T2$cluster==2])
cl2_x2 <- mean(T2$X2[T2$cluster==2])
cl1 <- c(cl1_x1,cl1_x2)
cl2 <- c(cl2_x1,cl2_x2)
cl1

## [1] 3.0 2.5

cl2

## [1] 2.75 2.25
```

d) Assign each observation to the centroid to which it is closest, in terms of Euclidean distance. Report the cluster labels for each observation.

```
d1 <- ((T2$X1-cl1_x1)^2 + (T2$X2-cl1_x2)^2)^(1/2)  ## distance to cluster 1 c
entroid
d2 <- ((T2$X1-cl2_x1)^2 + (T2$X2-cl2_x2)^2)^(1/2)  ## distance to cluster 2 c
entroid

#assigning the new cluster to the smallest distance
T2 <- cbind.data.frame(T2, d1, d2, new_cluster1 = c(2,2,2,1,1,2) )
T2

##   Obs. X1 X2 cluster       d1       d2 new_cluster1
## 1    1  1  4       1 2.500000 2.474874            2
## 2    2  1  3       2 2.061553 1.903943            2
## 3    3  0  4       2 3.354102 3.259601            2
## 4    4  5  1       1 2.500000 2.573908            1
```

```
## 5    5  6  2        2 3.041381 3.259601            1
## 6    6  4  0        2 2.692582 2.573908            2
```

e) Repeat (c) and (d) until the answers obtained stop changing.

```
# new centroids
ncl1_x1 <- mean(T2$X1[T2$new_cluster1==1])
ncl1_x2 <- mean(T2$X2[T2$new_cluster1==1])
ncl2_x1 <- mean(T2$X1[T2$new_cluster1==2])
ncl2_x2 <- mean(T2$X2[T2$new_cluster1==2])
ncl1 <- c(ncl1_x1,ncl1_x2)
ncl2 <- c(ncl2_x1,ncl2_x2)

# new cluster label assignment in table T2
d1 <- ((T2$X1-ncl1_x1)^2 + (T2$X2-ncl1_x2)^2)^(1/2)  ## distance to cluster 1
centroid
d2 <- ((T2$X1-ncl2_x1)^2 + (T2$X2-ncl2_x2)^2)^(1/2)  ## distance to cluster 2
centroid

T2 <- cbind.data.frame(T2[,c(1:3,7)], d1, d2, new_cluster2 = c(2,2,2,1,1,1) )
T2

##   Obs. X1 X2 new_cluster1        d1       d2 new_cluster2
## 1    1  1  4            2 5.1478151 1.346291            2
## 2    2  1  3            2 4.7434165 0.559017            2
## 3    3  0  4            2 6.0415230 1.952562            2
## 4    4  5  1            1 0.7071068 3.913119            1
## 5    5  6  2            1 0.7071068 4.562072            1
## 6    6  4  0            2 2.1213203 3.716517            1
```

We still have differences on last observation clustering, so next iteration is below

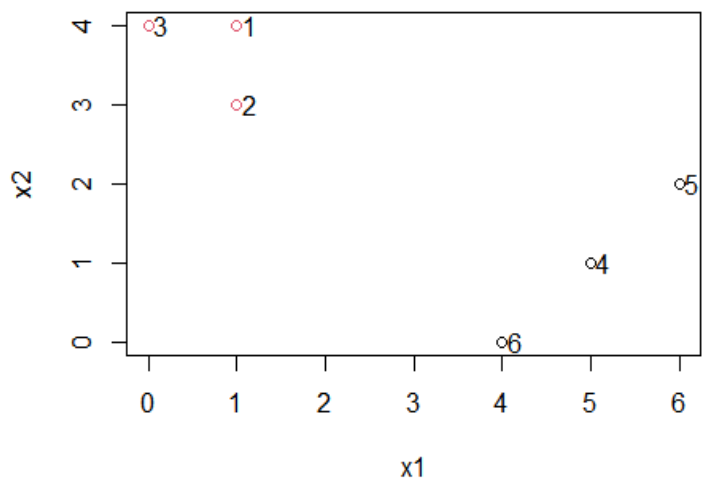
```
# new centroids
nncl1 <- c(mean(T2$X1[T2$new_cluster2==1]),mean(T2$X2[T2$new_cluster2==1]))
nncl2 <- c(mean(T2$X1[T2$new_cluster2==2]),mean(T2$X2[T2$new_cluster2==2]))

# new cluster label assignment in table T2
d1 <- ((T2$X1-nncl1[1])^2 + (T2$X2-nncl1[2])^2)^(1/2)  ## distance to cluster
1 centroid
d2 <- ((T2$X1-nncl2[1])^2 + (T2$X2-nncl2[2])^2)^(1/2)  ## distance to cluster
2 centroid

T2 <- cbind.data.frame(T2[,c(1:3,7)], d1, d2, new_cluster3 = c(2,2,2,1,1,1) )
T2

##   Obs. X1 X2 new_cluster2       d1        d2 new_cluster3
## 1    1  1  4            2 5.000000 0.4714045            2
## 2    2  1  3            2 4.472136 0.7453560            2
## 3    3  0  4            2 5.830952 0.7453560            2
## 4    4  5  1            1 0.000000 5.0881125            1
```

```
## 5     5  6  2            1 1.414214 5.5876849            1
## 6     6  4  0            1 1.414214 4.9553562            1
```

There are no more changes in clustering assignment, so results are: - Cluster 1: observations 4,5,6. Centroid is (5,1) - Cluster 2: observations 1,2,3. Centroid is (0.7,3.7)

f) In your plot from (a), color the observations according to the cluster labels obtained.

```
plot(T2$X1,T2$X2, xlab="x1", ylab="x2", col=T2$new_cluster3)
text(T2$X1+0.15,T2$X2,T2$Obs.)
```



In red the observations that correspond to cluster 2, and in black the observations in cluster 1.

Problem 4

Suppose that for a particular data set, we perform hierarchical clustering using single linkage and using complete linkage. We obtain two dendrograms. Answers for questions a and b are below:

a) {1,2,3} and {4,5} fuse in both dendograms. We can expect that the fusion in complete linkage will occur at a higher point of the tree than the one using single linkage as the complete linkage uses the highest dissimilarity/distance. However, it is possible that they occur also at the same height in the case that the distances are the same (then, there will be no max or min distance to choose).

b) {5} and {6} fuse in both dendograms. As these are single observations (and no clustered previously fused observations) the distance/dissimilarity will be the same as the linkage is not relevant. So, the height of this fusion in both trees will be the same.

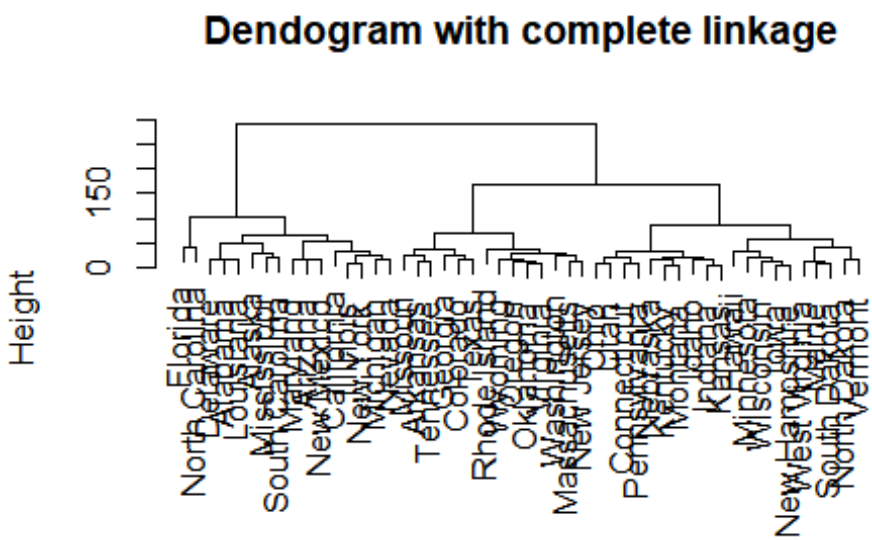
Problem 9

Consider the USArrests data. We will now perform hierarchical clustering on the states.

```
data("USArrests")
names(USArrests)

## [1] "Murder"   "Assault"  "UrbanPop" "Rape"
```

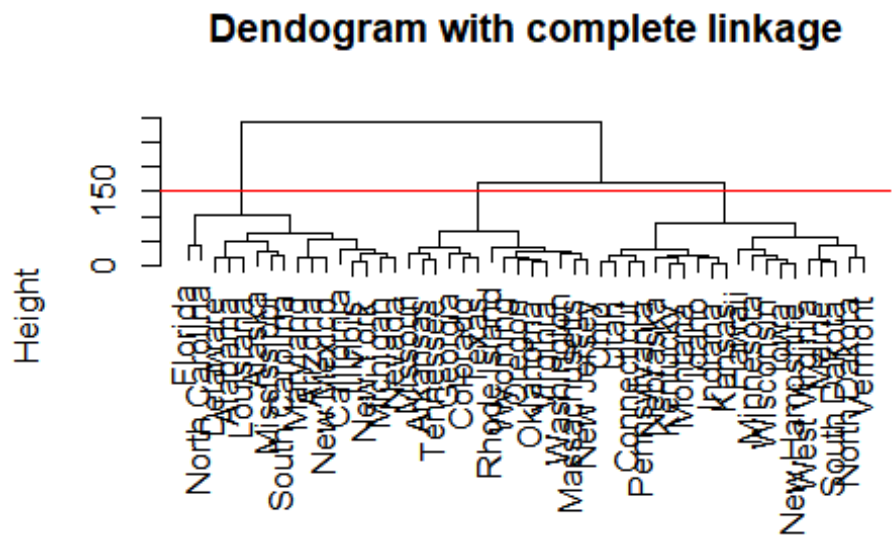
a) Using hierarchical clustering with complete linkage and Euclidean distance, cluster the states.

```
hc_cp <- hclust(dist(USArrests), method = "complete")
plot(hc_cp, main = "Dendogram with complete linkage", xlab="")
```



b) Cut the dendrogram at a height that results in three distinct clusters. Which states belong to which clusters? If we cut the dendogram at a height of 150, we get three clusters. The corresponding clusters for each state are shown below.

```
plot(hc_cp, main = "Dendogram with complete linkage", xlab="")
abline(h = 150, col="red")
```

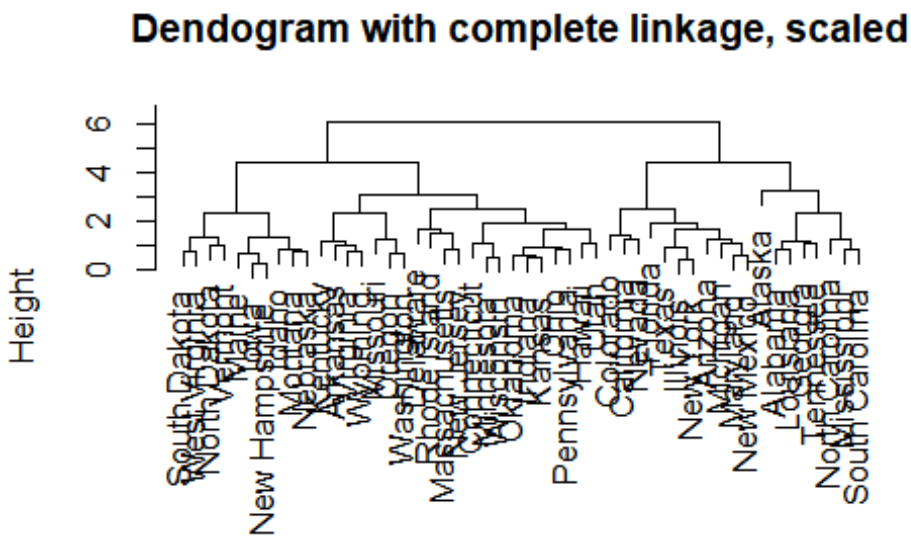



```
cutree(hc_cp,3)

##        Alabama         Alaska        Arizona       Arkansas     California
##              1              1              1              2              1
##       Colorado    Connecticut       Delaware        Florida        Georgia
##              2              3              1              1              2
##         Hawaii          Idaho       Illinois        Indiana           Iowa
##              3              3              1              3              3
##         Kansas       Kentucky      Louisiana          Maine       Maryland
##              3              3              1              3              1
##  Massachusetts       Michigan      Minnesota    Mississippi       Missouri
##              2              1              3              1              2
##        Montana       Nebraska         Nevada  New Hampshire     New Jersey
##              3              3              1              3              2
##     New Mexico       New York North Carolina   North Dakota           Ohio
##              1              1              1              3              3
##       Oklahoma         Oregon   Pennsylvania   Rhode Island South Carolina
##              2              2              3              2              1
##   South Dakota      Tennessee          Texas           Utah        Vermont
##              3              2              2              3              3
##       Virginia     Washington  West Virginia      Wisconsin        Wyoming
##              2              2              3              3              2
```

c)Hierarchically cluster the states using complete linkage and Euclidean distance, after scaling the variables to have standard deviation one.

```
scl <- scale(USArrests)
var(scl) # here we check that the variance is one for all variables, in the d
iagonal of the matrix

##              Murder   Assault   UrbanPop      Rape
## Murder   1.00000000 0.8018733 0.06957262 0.5635788
## Assault  0.80187331 1.0000000 0.25887170 0.6652412
```

```
## UrbanPop 0.06957262 0.2588717 1.00000000 0.4113412
## Rape     0.56357883 0.6652412 0.41134124 1.0000000

hc_cp_sc <- hclust(dist(scl), method = "complete")
plot(hc_cp_sc, main = "Dendogram with complete linkage, scaled", xlab="")
```



d) What effect does scaling the variables have on the hierarchical clustering obtained? In your opinion, should the variables be scaled before the inter-observation dissimilarities are computed? Provide a justification for your answer.

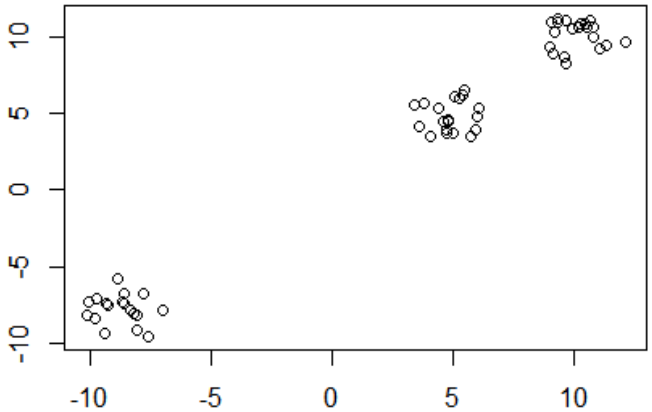
The dendogram obtained when scaling the variables is more balanced than the one that is not scaled. This means that we have less observations fused on a same cluster and the height among these fusions are more spread. In my opinion, the variables should be scaled before the inter-observation dissimilarities are computed, as we would be able to get more balanced clusters at the height we decide to make a cut in the dendogram.

Problem 10

In this problem, you will generate simulated data, and then perform PCA and K-means clustering on the data.

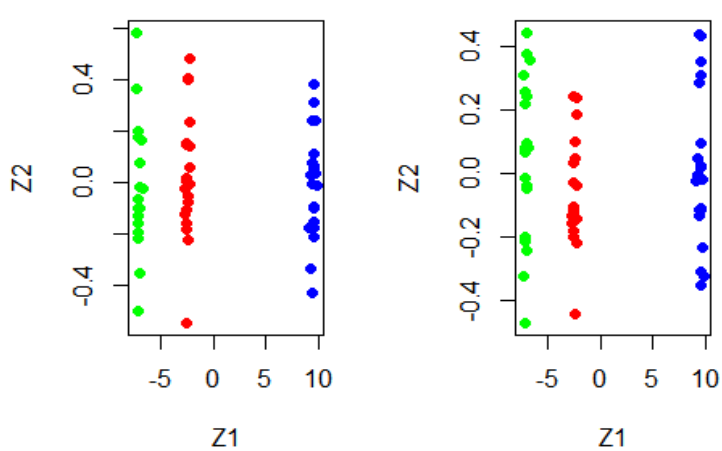
a) Generate a simulated data set with 20 observations in each of three classes (i.e. 60 observations total), and 50 variables.

```
set.seed(10)
ds <- matrix(rnorm(60*50), ncol=50)
ds[1:20,] <- ds[1:20,] + 5
ds[21:40,] <- ds[21:40,] - 8
ds[41:60,] <- ds[41:60,] + 10
plot(ds, xlab="", ylab="")
```



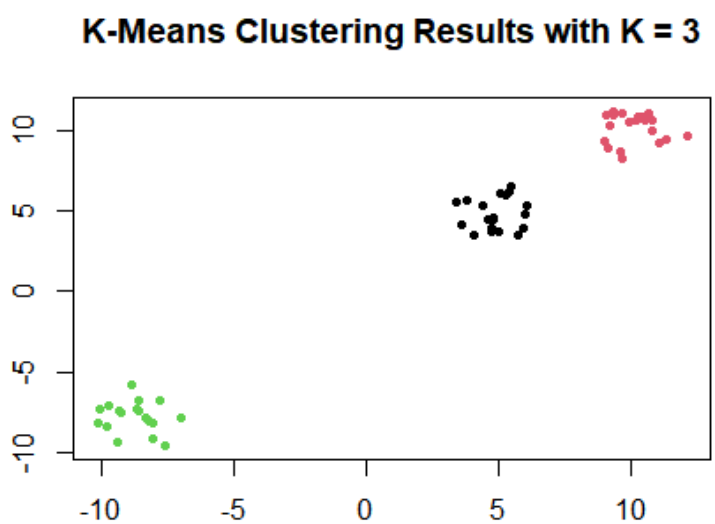
b) Perform PCA on the 60 observations and plot the first two principal component score vectors. Use a different color to indicate the observations in each of the three classes.

```
pca <- prcomp(ds, scale = TRUE)

# adding class labels to the data set
class <- data.frame(class = c(rep(1,20), rep(3,20), rep(2,20)))
ds_app <- data.frame(ds,class)

# function to color observations by class
Cols <- function(vec) {
  cols <-rainbow(length(unique(vec)))
  return(cols[as.numeric(as.factor(vec))])
}

# plot of first two principal components
par(mfrow = c(1, 2))
plot(pca$x[, 1:2], col = Cols(ds_app$class) , pch = 19, xlab = "Z1", ylab = "
Z2")
plot(pca$x[, c(1,3)], col = Cols(ds_app$class) , pch = 19, xlab = "Z1", ylab
= "Z2")
```



c) Perform K-means clustering of the observations with K = 3. How well do the clusters that you obtained in K-means clustering compare to the true class labels?

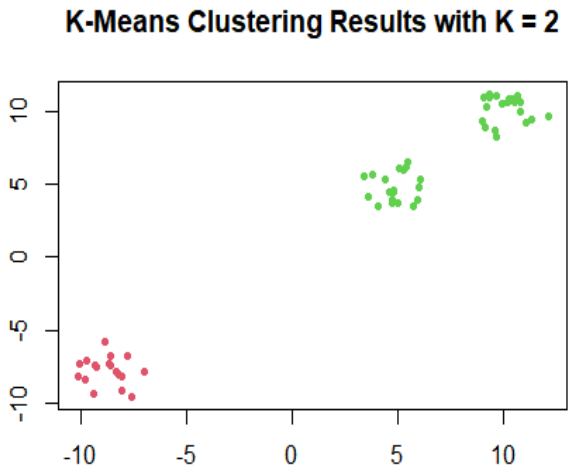
```
set.seed(3)
km_3 <- kmeans(ds, 3, nstart = 20)
table(true_class = ds_app$class, kmeans=km_3$cluster)

##            kmeans
## true_class  1  2  3
##          1 20  0  0
##          2  0 20  0
##          3  0  0 20

plot(ds, col = (km_3$cluster), main = "K-Means Clustering Results with K = 3"
, xlab = "", ylab = "", pch = 20)
```



As we can see from the table and the plot, using k-means with k=3 performs well the clustering when we compare to the true class labels. All the observations were accurately assigned to the same clusters.

d) Perform K-means clustering with K = 2. Describe your results.

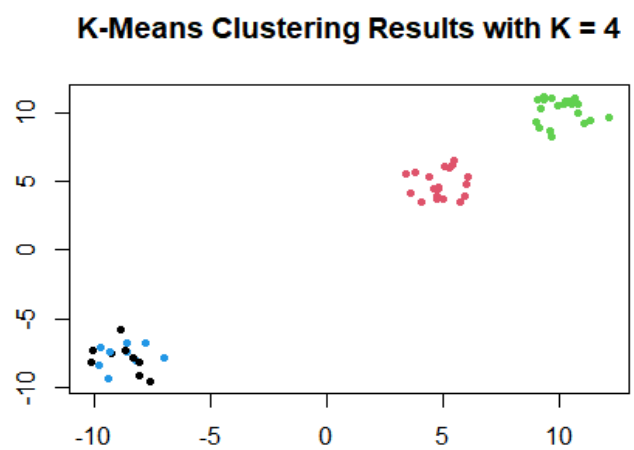
```
set.seed(2)
km_2 <- kmeans(ds, 2, nstart = 20)
table(true_class = ds_app$class, kmeans=km_2$cluster)

##            kmeans
## true_class  1  2
##          1  0 20
##          2  0 20
##          3 20  0

plot(ds, col = (km_2$cluster+1), main = "K-Means Clustering Results with K =
2", xlab = "", ylab = "", pch = 20)
```



As we can see, k-means clustering when setting k=2 is not clustering well. The true class label differs a lot with the k-means class obtained. We got observations that have a large dissimilarity grouped on the same cluster.

e) Now perform K-means clustering with K = 4, and describe your results.

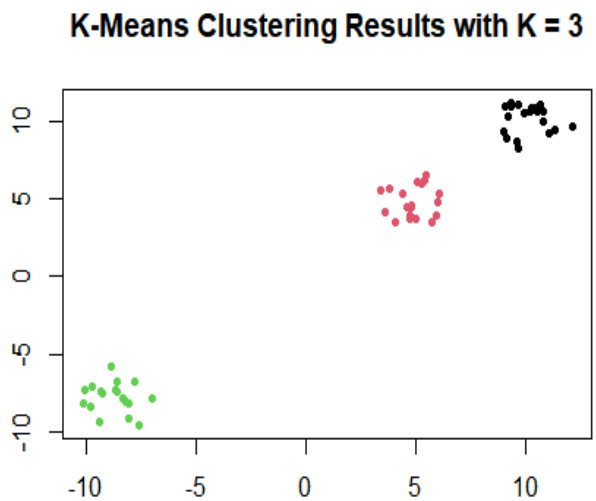
```
set.seed(4)
km_4 <- kmeans(ds, 4, nstart = 20)
table(true_class = ds_app$class, kmeans=km_4$cluster)

##            kmeans
## true_class  1  2  3  4
##          1  0 20  0  0
##          2  0  0 20  0
##          3 11  0  0  9

plot(ds, col = (km_4$cluster), main = "K-Means Clustering Results with K = 4"
, xlab = "", ylab = "", pch = 20)
```



In this case, when we have k=4 the clusters are not a good representation of the true class labels. We can see that from the table and the plot, where we have many observations that are close to each other (and indeed, belong to the same true class) and the k-means clustering is splitting them into two different clusters.

f) Now perform K-means clustering with K = 3 on the first two principal component score vectors, rather than on the raw data. Comment on the results.

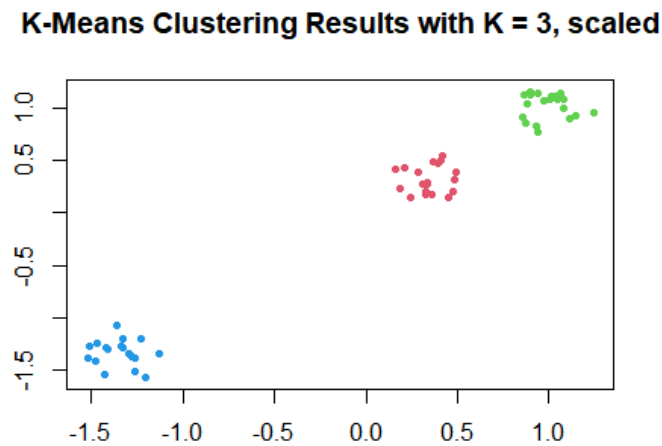
```
set.seed(31)
km_3_pca <- kmeans(pca$x[,1:2], 3, nstart = 20)
table(true_class = ds_app$class, kmeans=km_3_pca$cluster)

##            kmeans
## true_class  1  2  3
##          1  0 20  0
##          2 20  0  0
##          3  0  0 20

plot(ds, col = (km_3_pca$cluster), main = "K-Means Clustering Results with K
= 3", xlab = "", ylab = "", pch = 20)
```



The observations are well grouped together; however, they differ with the true class label. For example, class 1 observations are clustered as class 2, and class 2 as class 1.

g) Using the scale() function, perform K-means clustering with K = 3 on the data after scaling each variable to have standard deviation one. How do these results compare to those obtained in (b)? Explain.

```
set.seed(32)
km_3_sc <- kmeans(scale(ds), 3, nstart = 20)
table(true_class = ds_app$class, kmeans=km_3_sc$cluster)

##            kmeans
## true_class  1  2  3
##          1 20  0  0
##          2  0 20  0
##          3  0  0 20

plot(scale(ds), col = (km_3_sc$cluster+1), main = "K-Means Clustering Results
with K = 3, scaled", xlab = "", ylab = "", pch = 20)
```



The results obtained using the scale() function are the same, all the observations are clustered correctly compared to their true class labels.

Problem 11

Matrix completion function, based on Algorithm 12.4

```
MatComp <- function(DF,m){
  # step 1
  X <- data.matrix(DF)
  X_hat <- X
  ##missing values index
  ind <- which(is.na(X_hat))  # posiciones de la matrix
  x_bar <- colMeans(X_hat , na.rm = TRUE)
  aux <- matrix(rep(x_bar, nrow(X_hat)), nrow = nrow(X_hat), byrow=TRUE)
  X_hat[ind] <- aux[ind]

  # Step 2
  thresh <- 1e-7
  rel_err <- 1
  iter <- 0
  ismiss <- is.na(X)
  mssold <- mean ((scale(X , x_bar , FALSE)[!ismiss])^2)
  mss0 <- mean(X[!ismiss ]^2)
  max_iter <- 30

  fit.svd <- function(X, M = m) {
    svdob <- svd(X)
    with(svdob , u[, 1:M, drop = FALSE] %*% (d[1:M] * t(v[, 1:M, drop = FALSE
])))
  }

  err <- rep(NA, max_iter)
```

```r
  while(rel_err > thresh & iter < max_iter) {
    iter <- iter + 1
    # Step 2(a)
    Xapp <- fit.svd(X_hat , M = m)
    # Step 2(b)
    X_hat[ismiss] <- Xapp[ismiss]
    # Step 2(c)
    mss <- mean((( X - Xapp)[!ismiss ])^2)
    rel_err <- (mssold - mss) / mss0
    mssold <- mss
    cat("Iter:", iter, "MSS:", mss, "Rel. Err:", rel_err, "\n")
    err[iter] <- rel_err
  }
  err <- err[is.na(err) != TRUE]
  #print(err)
}
```

Test function on Boston data. - First, standardize features - Randomly leave out 5% of the observations, so we have a new matrix with missing values (NA).

```r
Bostonp <- as.matrix(Boston)
Bostonp <-  scale(Bostonp)

pc <- 0.05
omit <- round(pc*nrow(Bostonp)) # no. of obs to be removed
set.seed(11)
ina <- sample(seq(nrow(Bostonp)) , omit)
inb <- sample(seq(ncol(Bostonp)) , omit , replace=TRUE)
Xna <- Bostonp # creating aux matrix X with missing values
index_na <- cbind(ina,inb) # matrix of index to be set as "NA"
Xna[index_na] <- NA
```

- Apply Algorithm, the created function, with M=1,2,…,8. Display results.

```r
for (m in 2:8){
  cat("M = ", m, "\n")
  MatComp(Xna, m)
  cat("\n")
}
```

```
## M =  2
## Iter: 1 MSS: 0.3868237 Rel. Err: 0.6131063
## Iter: 2 MSS: 0.3864226 Rel. Err: 0.0004012231
## Iter: 3 MSS: 0.3864008 Rel. Err: 2.17987e-05
## Iter: 4 MSS: 0.3863993 Rel. Err: 1.475726e-06
## Iter: 5 MSS: 0.3863992 Rel. Err: 1.078081e-07
## Iter: 6 MSS: 0.3863992 Rel. Err: 8.093086e-09
##
## M =  3
## Iter: 1 MSS: 0.2898741 Rel. Err: 0.7100723
## Iter: 2 MSS: 0.2893763 Rel. Err: 0.0004979473
```

```
## Iter: 3 MSS: 0.2893387 Rel. Err: 3.759264e-05
## Iter: 4 MSS: 0.2893353 Rel. Err: 3.401954e-06
## Iter: 5 MSS: 0.2893349 Rel. Err: 3.33496e-07
## Iter: 6 MSS: 0.2893349 Rel. Err: 3.415147e-08
##
## M =  4
## Iter: 1 MSS: 0.2221438 Rel. Err: 0.7778141
## Iter: 2 MSS: 0.2216412 Rel. Err: 0.0005026807
## Iter: 3 MSS: 0.2215906 Rel. Err: 5.057477e-05
## Iter: 4 MSS: 0.2215828 Rel. Err: 7.829606e-06
## Iter: 5 MSS: 0.2215804 Rel. Err: 2.419614e-06
## Iter: 6 MSS: 0.221579 Rel. Err: 1.339625e-06
## Iter: 7 MSS: 0.2215781 Rel. Err: 9.125204e-07
## Iter: 8 MSS: 0.2215775 Rel. Err: 6.521337e-07
## Iter: 9 MSS: 0.221577 Rel. Err: 4.710673e-07
## Iter: 10 MSS: 0.2215766 Rel. Err: 3.412586e-07
## Iter: 11 MSS: 0.2215764 Rel. Err: 2.475113e-07
## Iter: 12 MSS: 0.2215762 Rel. Err: 1.796531e-07
## Iter: 13 MSS: 0.2215761 Rel. Err: 1.304799e-07
## Iter: 14 MSS: 0.221576 Rel. Err: 9.481789e-08
##
## M =  5
## Iter: 1 MSS: 0.1597898 Rel. Err: 0.8401787
## Iter: 2 MSS: 0.159283 Rel. Err: 0.0005068242
## Iter: 3 MSS: 0.1592178 Rel. Err: 6.527242e-05
## Iter: 4 MSS: 0.1592045 Rel. Err: 1.328081e-05
## Iter: 5 MSS: 0.1592002 Rel. Err: 4.340727e-06
## Iter: 6 MSS: 0.1591982 Rel. Err: 1.907777e-06
## Iter: 7 MSS: 0.1591973 Rel. Err: 9.488041e-07
## Iter: 8 MSS: 0.1591968 Rel. Err: 4.967992e-07
## Iter: 9 MSS: 0.1591965 Rel. Err: 2.692108e-07
## Iter: 10 MSS: 0.1591964 Rel. Err: 1.518466e-07
## Iter: 11 MSS: 0.1591963 Rel. Err: 9.06819e-08
##
## M =  6
## Iter: 1 MSS: 0.1184983 Rel. Err: 0.8814771
## Iter: 2 MSS: 0.1179111 Rel. Err: 0.0005873875
## Iter: 3 MSS: 0.1178299 Rel. Err: 8.120571e-05
## Iter: 4 MSS: 0.1178132 Rel. Err: 1.662975e-05
## Iter: 5 MSS: 0.1178079 Rel. Err: 5.34458e-06
## Iter: 6 MSS: 0.1178055 Rel. Err: 2.357298e-06
## Iter: 7 MSS: 0.1178043 Rel. Err: 1.238675e-06
## Iter: 8 MSS: 0.1178036 Rel. Err: 7.334004e-07
## Iter: 9 MSS: 0.1178031 Rel. Err: 4.852195e-07
## Iter: 10 MSS: 0.1178027 Rel. Err: 3.583056e-07
## Iter: 11 MSS: 0.1178024 Rel. Err: 2.91679e-07
## Iter: 12 MSS: 0.1178022 Rel. Err: 2.557658e-07
## Iter: 13 MSS: 0.1178019 Rel. Err: 2.356951e-07
## Iter: 14 MSS: 0.1178017 Rel. Err: 2.238552e-07
## Iter: 15 MSS: 0.1178015 Rel. Err: 2.163136e-07
```

```
## Iter: 16 MSS: 0.1178013 Rel. Err: 2.110286e-07
## Iter: 17 MSS: 0.1178011 Rel. Err: 2.069372e-07
## Iter: 18 MSS: 0.1178009 Rel. Err: 2.034849e-07
## Iter: 19 MSS: 0.1178007 Rel. Err: 2.003823e-07
## Iter: 20 MSS: 0.1178005 Rel. Err: 1.97478e-07
## Iter: 21 MSS: 0.1178003 Rel. Err: 1.94693e-07
## Iter: 22 MSS: 0.1178001 Rel. Err: 1.919857e-07
## Iter: 23 MSS: 0.1177999 Rel. Err: 1.893344e-07
## Iter: 24 MSS: 0.1177997 Rel. Err: 1.867275e-07
## Iter: 25 MSS: 0.1177995 Rel. Err: 1.841589e-07
## Iter: 26 MSS: 0.1177994 Rel. Err: 1.816254e-07
## Iter: 27 MSS: 0.1177992 Rel. Err: 1.79125e-07
## Iter: 28 MSS: 0.117799 Rel. Err: 1.766566e-07
## Iter: 29 MSS: 0.1177988 Rel. Err: 1.742196e-07
## Iter: 30 MSS: 0.1177986 Rel. Err: 1.718133e-07
##
## M =  7
## Iter: 1 MSS: 0.08747349 Rel. Err: 0.9125072
## Iter: 2 MSS: 0.08680886 Rel. Err: 0.000664737
## Iter: 3 MSS: 0.0866762 Rel. Err: 0.000132687
## Iter: 4 MSS: 0.08663511 Rel. Err: 4.109425e-05
## Iter: 5 MSS: 0.08661774 Rel. Err: 1.737982e-05
## Iter: 6 MSS: 0.08660919 Rel. Err: 8.54792e-06
## Iter: 7 MSS: 0.08660466 Rel. Err: 4.52944e-06
## Iter: 8 MSS: 0.08660214 Rel. Err: 2.522214e-06
## Iter: 9 MSS: 0.08660067 Rel. Err: 1.466154e-06
## Iter: 10 MSS: 0.08659978 Rel. Err: 8.890778e-07
## Iter: 11 MSS: 0.08659922 Rel. Err: 5.63496e-07
## Iter: 12 MSS: 0.08659885 Rel. Err: 3.745988e-07
## Iter: 13 MSS: 0.08659858 Rel. Err: 2.623063e-07
## Iter: 14 MSS: 0.08659839 Rel. Err: 1.941371e-07
## Iter: 15 MSS: 0.08659824 Rel. Err: 1.519911e-07
## Iter: 16 MSS: 0.08659811 Rel. Err: 1.255006e-07
## Iter: 17 MSS: 0.086598 Rel. Err: 1.085817e-07
## Iter: 18 MSS: 0.08659791 Rel. Err: 9.759013e-08
##
## M =  8
## Iter: 1 MSS: 0.06593756 Rel. Err: 0.9340468
## Iter: 2 MSS: 0.0652801 Rel. Err: 0.0006575742
## Iter: 3 MSS: 0.0650979 Rel. Err: 0.0001822316
## Iter: 4 MSS: 0.06502774 Rel. Err: 7.016959e-05
## Iter: 5 MSS: 0.06499319 Rel. Err: 3.456224e-05
## Iter: 6 MSS: 0.06497292 Rel. Err: 2.027236e-05
## Iter: 7 MSS: 0.06495953 Rel. Err: 1.339044e-05
## Iter: 8 MSS: 0.06494998 Rel. Err: 9.549801e-06
## Iter: 9 MSS: 0.06494284 Rel. Err: 7.142279e-06
## Iter: 10 MSS: 0.06493734 Rel. Err: 5.498632e-06
## Iter: 11 MSS: 0.06493303 Rel. Err: 4.309091e-06
## Iter: 12 MSS: 0.06492962 Rel. Err: 3.414861e-06
## Iter: 13 MSS: 0.06492689 Rel. Err: 2.726115e-06
```

```
## Iter: 14 MSS: 0.06492471 Rel. Err: 2.187317e-06
## Iter: 15 MSS: 0.06492295 Rel. Err: 1.761508e-06
## Iter: 16 MSS: 0.06492152 Rel. Err: 1.422671e-06
## Iter: 17 MSS: 0.06492037 Rel. Err: 1.151739e-06
## Iter: 18 MSS: 0.06491944 Rel. Err: 9.343355e-07
## Iter: 19 MSS: 0.06491868 Rel. Err: 7.594119e-07
## Iter: 20 MSS: 0.06491806 Rel. Err: 6.183633e-07
## Iter: 21 MSS: 0.06491756 Rel. Err: 5.044259e-07
## Iter: 22 MSS: 0.06491714 Rel. Err: 4.122479e-07
## Iter: 23 MSS: 0.06491681 Rel. Err: 3.375738e-07
## Iter: 24 MSS: 0.06491653 Rel. Err: 2.770072e-07
## Iter: 25 MSS: 0.0649163 Rel. Err: 2.278294e-07
## Iter: 26 MSS: 0.06491611 Rel. Err: 1.878588e-07
## Iter: 27 MSS: 0.06491596 Rel. Err: 1.553413e-07
## Iter: 28 MSS: 0.06491583 Rel. Err: 1.288639e-07
## Iter: 29 MSS: 0.06491572 Rel. Err: 1.072865e-07
## Iter: 30 MSS: 0.06491563 Rel. Err: 8.968798e-08
```

From the above example, we can see that for larger values of M, we would need more iterations to reach that minimum objective function. When M=1 there are only 6 iterations, while with M=8 we have 30 iterations and still have not reached the minimum threshold.

STAT 508 - HOMEWORK #5

Problem 2

- Dissimilarity Matrix $\rightarrow$

$$\begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 0.3 & 0.4 & 0.7 \\ 2 & 0.3 & 0 & 0.5 & 0.8 \\ 3 & 0.4 & 0.5 & 0 & 0.45 \\ 4 & 0.7 & 0.8 & 0.45 & 0 \end{array}$$

a) Dendogram with complete linkage

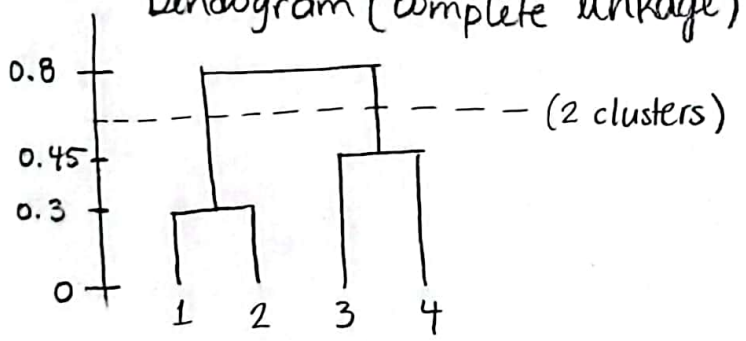
Fuse 1-2 $\rightarrow$

$$\begin{array}{c|ccc} & (12) & 3 & 4 \\ \hline (12) & 0 & 0.5 & 0.8 \\ 3 & 0.5 & 0 & 0.45 \\ 4 & 0.8 & 0.45 & 0 \end{array}$$

Fuse 3-4 $\rightarrow$

$$\begin{array}{c|cc} & (12) & (34) \\ \hline (12) & 0 & 0.8 \\ (34) & 0.8 & 0 \end{array}$$

Dendogram (complete linkage)



b) Dendogram with single linkage

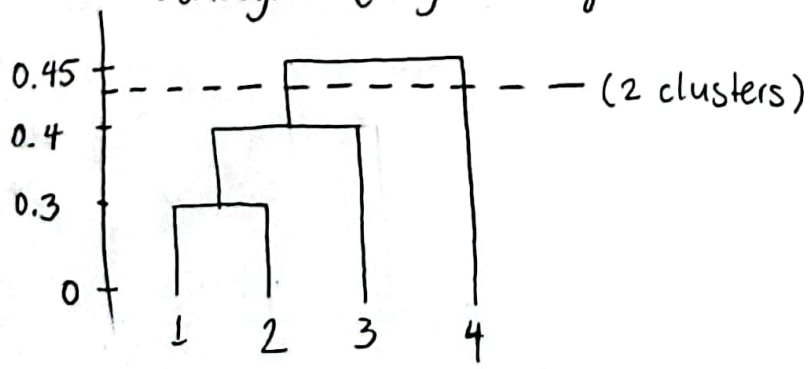
Fuse 1-2 $\rightarrow$

$$\begin{array}{c|ccc} & (12) & 3 & 4 \\ \hline (12) & 0 & 0.4 & 0.7 \\ 3 & 0.4 & 0 & 0.45 \\ 4 & 0.7 & 0.45 & 0 \end{array}$$

Fuse 3-(12) $\rightarrow$

$$\begin{array}{c|cc} & (123) & 4 \\ \hline (123) & 0 & 0.45 \\ 4 & 0.45 & 0 \end{array}$$

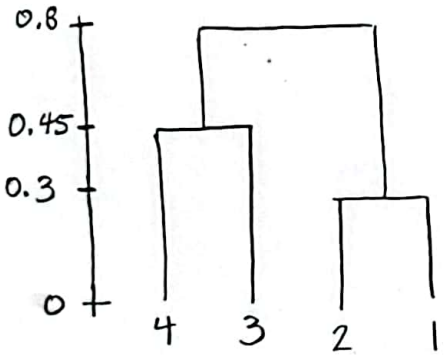
Dendogram (single linkage)



c) If we cut Dendogram (a) to get 2 clusters they will be:

* Cluster 1 → Obs. 1 and 2
* Cluster 2 → Obs. 3 and 4

d) If we cut Dendogram (b) to get 2 clusters they will be:

* Cluster 1 → Obs. 1, 2, 3
* Cluster 2 → Obs. 4

e) Equivalent Dendogram for (a)